SECTION 1 — Databricks Platform Fundamentals

Q1. Explain the Databricks Lakehouse architecture and how it differs from traditional data warehouse architecture.

The Databricks Lakehouse architecture is designed to combine the benefits of data lakes and data warehouses into a single unified platform.

In traditional data architectures, organizations often use separate systems for storage and analytics. Raw data is stored in data lakes, which are cost-effective and scalable but lack strong reliability features. On the other hand, structured analytics workloads run in data warehouses, which provide high performance and reliability but are expensive and less flexible.

The lakehouse architecture solves this problem by allowing both raw and structured data to be stored in the same cloud storage system while providing database-like features such as transactions, schema enforcement, and versioning.

In Databricks, the storage layer usually consists of cloud object storage like Amazon S3, Azure Data Lake Storage (ADLS), or Google Cloud Storage. On top of this storage layer sits Delta Lake, which provides ACID transactions and reliability. The compute layer consists of Spark clusters that process and transform data.

This architecture allows organizations to run multiple workloads including ETL pipelines, machine learning models, and business intelligence queries on the same platform.

Example:

An e-commerce company collects data from multiple sources including orders, user clicks, and product inventory. Instead of storing raw data in one system and analytics data in another, all the data is stored in cloud storage and managed using Delta Lake tables. Engineers process this data using Spark clusters and generate business metrics for dashboards.

Key takeaway:
Lakehouse architecture reduces data duplication, improves reliability, and enables unified analytics on large-scale data.

Q2. Explain Bronze, Silver, and Gold architecture in Databricks pipelines.

The Bronze, Silver, and Gold architecture is a layered approach used in data engineering pipelines to organize data processing stages and improve reliability.

The Bronze layer is the raw data layer. This layer stores data exactly as it arrives from the source systems without significant transformations. It acts as the source of truth and provides an audit trail. If downstream transformations fail, engineers can always reprocess data from this layer.

The Silver layer contains cleaned and validated data. In this layer, data engineers perform operations such as removing duplicates, correcting data types, handling missing values, and standardizing formats. The goal of this layer is to create a trusted dataset that can be used by internal teams.

The Gold layer contains aggregated or business-ready datasets. These datasets are optimized for analytics and reporting. They often contain metrics such as daily revenue, top-selling products, or customer lifetime value.

Example:

Consider a food delivery platform.

Bronze layer stores raw order events coming from the application.

Silver layer processes these events and produces clean tables with validated order records.

Gold layer generates analytical datasets such as daily revenue by city or top restaurants by sales.

Key takeaway:
Layered architecture improves pipeline reliability, debugging capability, and data quality.

Q3. What are the major components of the Databricks platform?

Databricks is a comprehensive data platform that provides multiple components for building and managing data pipelines.

The workspace is the development environment where engineers write notebooks, manage code repositories, and configure jobs.

Clusters are the compute engines that run Spark workloads. They consist of a driver node and multiple worker nodes responsible for processing data.

Notebooks are interactive environments where engineers write code in Python, SQL, Scala, or R to analyze and process data.

Delta Lake is the storage layer that provides transactional reliability, schema enforcement, and versioning on top of cloud object storage.

Unity Catalog is the governance layer responsible for managing permissions, access control, and data lineage across the platform.

Workflows are used to orchestrate pipelines by scheduling jobs and managing task dependencies.

Example:

A typical pipeline may involve a workflow triggering a job cluster that runs notebooks which transform raw data and store the results in Delta tables governed by Unity Catalog.

Key takeaway:
Databricks integrates compute, storage, governance, and orchestration into a single unified platform.

Q4. What is the difference between DBFS and cloud storage like S3 or ADLS?

DBFS stands for Databricks File System. It is an abstraction layer that allows Databricks to access files stored in cloud object storage.

The actual data is stored in cloud storage systems such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. DBFS simply provides a convenient interface for accessing those files within Databricks.

When users read or write files using DBFS paths, Databricks internally interacts with the underlying cloud storage.

Example:

A file stored in S3 might be accessed through a DBFS path in a notebook, allowing users to interact with it using Spark APIs.

Key takeaway:
DBFS is an interface layer, while the real persistent storage resides in cloud object storage.

Q5. How does Databricks separate compute and storage?

Databricks separates compute and storage to enable scalability and cost efficiency.

In this architecture, data is stored permanently in cloud storage, while compute clusters are created temporarily to process the data.

When a job runs, Databricks spins up a cluster to execute the workload. After the job completes, the cluster can be terminated without affecting the stored data.

This separation allows organizations to scale compute resources independently of storage.

Example:

A nightly ETL pipeline may start a cluster at 2 AM to process large datasets and terminate the cluster after completion. The data remains stored in cloud storage and can be accessed later by analytics tools.

Key takeaway:
Compute resources can scale dynamically while storage remains persistent and cost-effective.

Q6. Explain the typical data pipeline flow in Databricks.

A typical Databricks pipeline begins with data ingestion from various sources such as APIs, databases, or streaming platforms.

The ingested data is stored in the Bronze layer as raw data. This ensures that the original data is preserved.

Next, transformation processes clean and standardize the data, creating the Silver layer. These transformations may include deduplication, type conversions, and validation.

Finally, aggregated datasets are generated in the Gold layer for business analytics and reporting.

Workflows orchestrate the entire pipeline by scheduling tasks and ensuring that each stage runs in the correct order.

Example:

An online retailer may ingest order data into Bronze tables, clean and enrich the data in Silver tables, and generate daily revenue metrics in Gold tables for dashboards.

Key takeaway:
Databricks pipelines follow a structured flow from raw ingestion to analytics-ready datasets.

SECTION 2 — Clusters & Compute

Q7. What is the difference between Job Clusters and All-Purpose Clusters?

Job clusters are created specifically for running automated jobs and are terminated once the job completes. They are designed for production pipelines and provide a clean environment for each execution.

All-purpose clusters are long-running clusters used for development and exploration. Multiple users can run notebooks on these clusters interactively.

Example:

A data engineer testing transformation logic may use an all-purpose cluster, while a scheduled ETL pipeline would run on a job cluster.

Key takeaway:
Job clusters ensure predictable and isolated execution for production workloads.

Q8. Which cluster configuration parameters affect Spark performance?

Spark performance depends heavily on cluster configuration parameters such as the number of worker nodes, executor memory, CPU cores, and runtime version.

Increasing the number of worker nodes improves parallel processing capability. Memory allocation affects how much data can be processed without spilling to disk.

Example:

A cluster with 2 workers may take 40 minutes to process a dataset, while a cluster with 10 workers can complete the same workload in 10 minutes.

Key takeaway:
Proper cluster configuration is essential for achieving optimal performance in distributed workloads.

Q9. How does autoscaling work in Databricks clusters?

Autoscaling automatically adjusts the number of worker nodes in a cluster based on workload demand.

When the workload increases, additional worker nodes are added to handle the processing load. When the workload decreases, nodes are removed to reduce cost.

Example:

A pipeline processing large datasets during peak hours may scale from 2 workers to 8 workers automatically.

Key takeaway:
Autoscaling helps balance performance and cost in dynamic workloads.

Q10. Why should production pipelines avoid interactive clusters?

Interactive clusters are shared environments where multiple users run queries and experiments. This can lead to unpredictable performance and environment changes.

Production pipelines require stable and reproducible environments. Job clusters provide this reliability by creating a fresh cluster for each execution.

Example:

If an analyst runs a heavy query on an interactive cluster, it may slow down an ETL pipeline running on the same cluster.

Key takeaway:
Production workloads should run on dedicated job clusters for stability.

Q11. How can engineers reduce Databricks compute cost?

Compute cost can be reduced by optimizing cluster usage, processing data incrementally, and improving data layout.

Using job clusters with auto-termination ensures clusters run only when needed.

Incremental pipelines process only new data instead of reprocessing the entire dataset.

File compaction reduces query overhead by merging small files into larger ones.

Example:

An organization processing full datasets daily reduced costs by switching to incremental processing and optimized file layouts.

Key takeaway:
Efficient cluster management and incremental processing significantly reduce compute costs.

SECTION 3 — Delta Lake

Q12. What problems does Delta Lake solve compared to traditional data lakes?

Traditional data lakes store files in formats like Parquet or JSON but lack transactional guarantees. This means concurrent writes can corrupt data and schema inconsistencies can break queries.

Delta Lake introduces ACID transactions, schema enforcement, and versioning to address these issues.

Example:

If two pipelines write to the same table simultaneously, Delta Lake ensures that the table remains consistent.

Key takeaway:
Delta Lake transforms unreliable file-based data lakes into reliable data platforms.

Q13. Explain how Delta transaction logs work.

Delta tables maintain a transaction log stored in a directory called _delta_log.

Every change to the table is recorded as a log entry describing files added or removed. When a query runs, Spark reads the transaction log to determine the latest snapshot of the table.

This mechanism enables features such as time travel and concurrent writes.

Example:

If an update operation modifies records, Delta writes new data files and updates the transaction log instead of modifying existing files.

Key takeaway:
Transaction logs provide consistent snapshots of Delta tables.

Q14. Explain MERGE operation in Delta Lake.

MERGE allows updating existing rows and inserting new rows in a single operation. This is commonly used in incremental pipelines.

Example:

If new order data arrives containing both new and updated records, a MERGE statement can update existing records and insert new ones.

Key takeaway:
MERGE enables efficient upsert operations in Delta tables.

Q15. What is Time Travel in Delta Lake?

Time travel allows querying historical versions of a Delta table using version numbers or timestamps.

This is useful for debugging pipelines, auditing changes, and reproducing historical reports.

Example:

A financial analyst may query the version of a table from last week to verify revenue calculations.

Key takeaway:
Time travel provides built-in data versioning.

Q16. What are OPTIMIZE and ZORDER?

OPTIMIZE compacts small files into larger ones to improve query performance.

ZORDER organizes data files based on commonly filtered columns to enable data skipping.

Example:

If queries frequently filter by customer_id, ZORDER can cluster similar values together to reduce the number of scanned files.

Key takeaway:
File compaction and clustering improve query performance.

Q17. What is VACUUM and what risks exist?

VACUUM removes obsolete files that are no longer referenced by the Delta transaction log.

This helps reduce storage costs but must be used carefully because removing files too early can break time travel functionality.

Example:

Organizations often retain historical data for 7 days before running VACUUM.

Key takeaway:
VACUUM manages storage cleanup while preserving recent table versions.

SECTION 4 — Data Engineering Best Practices

Q18. How do you design incremental data pipelines in Databricks instead of full refresh pipelines?

In production data systems, reprocessing the entire dataset every time is inefficient and expensive. Incremental pipelines solve this problem by processing only the **new or changed data** since the last run.

The first step in designing an incremental pipeline is identifying a reliable column that can track new data. This is usually a **timestamp column**, an **auto-increment ID**, or a **change data capture field** such as `updated_at`.

The pipeline maintains a **checkpoint or watermark** which stores the latest processed value. During the next run, the pipeline reads only records that have values greater than the stored watermark.

This approach significantly reduces the amount of data processed and improves pipeline efficiency.

Example:

Suppose an e-commerce system generates **10 million order records daily**. If the pipeline processes the entire historical dataset every day, it will repeatedly scan billions of records.

Instead, an incremental pipeline reads only the new records created since the previous run.

Typical logic:

1. Retrieve the last processed timestamp
2. Read new records from source
3. Process and write results
4. Update the checkpoint

Example SQL logic:

SELECT *

```
FROM orders
WHERE order_timestamp > last_processed_timestamp
```

In Databricks pipelines, incremental processing is often implemented using **Delta MERGE operations**, **structured streaming**, or **Auto Loader**.

Key Takeaway:

Incremental pipelines improve performance, reduce cost, and allow systems to scale as data volume grows.

Q19. How do you handle late-arriving data in Delta Lake pipelines?

Late-arriving data refers to records that arrive after the expected processing window. This commonly occurs due to network delays, upstream system failures, or asynchronous event pipelines.

If pipelines assume that all data arrives on time, late records can cause incorrect aggregates or incomplete datasets.

To handle late-arriving data, engineers use techniques such as **event-time processing**, **watermarks**, and **upsert logic**.

One common strategy is to use **Delta MERGE operations** to update existing records when late data arrives.

Example:

Suppose an order event occurred on January 1st but the system receives the record on January 3rd due to system delays.

If the pipeline only processes data for the current day, the record would be missed.

Instead, the pipeline allows updates to historical data using MERGE operations.

Example:

```
MERGE INTO sales_table t
USING new_orders s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET *
```

WHEN NOT MATCHED THEN INSERT *

Another strategy is to reprocess a **small historical window** each day.

Example:

A pipeline may reprocess the last **3 days of data** to capture delayed events.

Key Takeaway:

Late-arriving data should be handled through **merge operations, reprocessing windows, or streaming watermarks** to ensure data accuracy.

Q20. How can engineers detect and fix the small files problem in distributed storage?

The small files problem occurs when a data lake contains a very large number of small files instead of fewer large files.

This issue reduces query performance because distributed processing engines like Spark must open and read metadata for every file.

If a table contains millions of small files, Spark spends significant time managing file metadata instead of processing data.

The problem typically arises when pipelines write data frequently in small batches.

Example:

A streaming pipeline writing files every few seconds may generate thousands of small files.

This leads to slower queries and inefficient resource usage.

The solution is **file compaction**, which merges many small files into fewer larger files.

In Delta Lake, engineers use the `OPTIMIZE` command.

Example:

OPTIMIZE sales_table

This operation combines small files into larger optimized files.

Another solution is controlling file size during writes using partitioning strategies and Spark configurations.

Example:

df.repartition(200)

Key Takeaway:

Reducing the number of files improves query performance and cluster efficiency.

Q21. How do you ensure idempotency in production Databricks pipelines?

Idempotency means that running a pipeline multiple times produces the same result.

This property is critical for production pipelines because failures or retries may cause jobs to run again.

Without idempotency, repeated runs can generate duplicate data.

The most common approach to achieve idempotency is using **MERGE operations** instead of simple inserts.

Example:

If an order pipeline fails after partially loading data, rerunning the pipeline should update existing records rather than inserting duplicates.

Example MERGE logic:

```
MERGE INTO target_table t
USING source_table s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Another technique is maintaining **checkpoint tables** that track processed batches.

For example:

A pipeline may store the last processed batch ID and skip previously processed data.

Example real-world scenario:

A payment processing system must avoid duplicate transactions. Idempotent pipelines guarantee consistent results even after retries.

Key Takeaway:

Idempotency ensures reliability by preventing duplicate data during pipeline retries.

Q22. How do you debug a slow Databricks job in production?

Debugging slow Spark jobs requires systematic analysis of execution metrics and pipeline design.

The first step is analyzing the **Spark UI**, which provides insights into job execution stages, task durations, and shuffle operations.

The Spark UI helps identify whether the slowdown is caused by skewed data, excessive shuffling, or inefficient joins.

Example:

If one task takes significantly longer than others, the issue may be data skew.

Another debugging step is reviewing **query execution plans** to understand how Spark processes the data.

For example, a query may trigger large shuffle operations due to poor partitioning.

Engineers also examine cluster resource usage such as CPU, memory, and disk I/O.

Example real-world case:

A pipeline processing 500 million rows may slow down because a single product generates 70 percent of all transactions, causing skewed partitions.

Possible solutions include repartitioning data or using broadcast joins.

Key Takeaway:

Effective debugging involves analyzing Spark UI metrics, query plans, and cluster resource utilization.

SECTION 5 — Governance & Security

Q23. What is Unity Catalog and why is it important for enterprise governance?

Unity Catalog is the centralized data governance system in Databricks. It provides a unified framework for managing data access, permissions, and metadata across an organization.

In large enterprises, multiple teams access the same datasets. Without proper governance, it becomes difficult to track who accessed the data and whether sensitive information is protected.

Unity Catalog solves this by organizing data into a hierarchy of **catalogs, schemas, and tables**.

Permissions can be assigned at different levels to control access.

Example:

A finance team may have access to financial datasets but not customer personally identifiable information.

Unity Catalog also provides **data lineage**, which tracks how datasets are created and transformed.

Example:

If a dashboard shows incorrect revenue numbers, engineers can trace the pipeline back to the source dataset.

Key Takeaway:

Unity Catalog enables centralized data governance, security, and lineage tracking.

Q24. How should secrets be securely managed in Databricks pipelines?

Production pipelines often require credentials to access external systems such as databases, APIs, or cloud storage.

Storing credentials directly in code is insecure because it exposes sensitive information.

Instead, Databricks uses **secret scopes** to securely store credentials.

These secrets can be retrieved during runtime without exposing them in notebooks.

Example:

A pipeline accessing an external database retrieves credentials from a secret scope rather than hardcoding them.

Example code:

```
dbutils.secrets.get(scope="prod-secrets", key="db-password")
```

Secret scopes are often integrated with external services such as **Azure Key Vault or AWS Secrets Manager**.

Key Takeaway:

Secure secret management prevents credential leakage and improves system security.

Q25. How does Databricks manage table permissions?

Databricks uses role-based access control through Unity Catalog.

Permissions can be assigned at multiple levels including catalogs, schemas, and tables.

Example permissions include:

SELECT permission for reading data

MODIFY permission for updating tables

CREATE permission for creating new objects

Example scenario:

A data analyst may have SELECT access to Gold tables but not access to raw Bronze datasets.

This layered access ensures sensitive data remains protected while allowing teams to perform analytics.

Key Takeaway:

Granular permissions protect sensitive data and enforce security policies.

SECTION 6 — Workflows

Q26. How do Databricks workflows help build reliable pipelines?

Databricks workflows provide built-in orchestration for scheduling and managing data pipelines.

A workflow consists of multiple tasks connected in a dependency structure.

Each task can execute a notebook, Python script, or SQL query.

Workflows also provide features such as retries, alerts, and failure handling.

Example:

A daily ETL pipeline may consist of three tasks:

1 Ingest raw data
2 Transform data into curated tables
3 Generate analytics datasets

If the transformation step fails, the workflow stops and alerts engineers.

Key Takeaway:

Workflows automate pipeline execution and improve reliability.

Q27. What are the advantages of Databricks workflows compared to external schedulers?

External schedulers such as Airflow or cron jobs can orchestrate pipelines but require additional infrastructure.

Databricks workflows provide native orchestration within the platform.

Advantages include simplified deployment, integrated monitoring, and easier debugging.

Example:

Instead of managing Airflow servers, teams can configure pipelines directly in Databricks.

Key Takeaway:

Native orchestration reduces infrastructure complexity.

SECTION 7 — Delta Live Tables

Q28. What is Delta Live Tables?

Delta Live Tables is a managed pipeline framework that simplifies building reliable data pipelines.

Instead of manually managing pipeline dependencies, engineers define transformations declaratively.

DLT automatically manages tasks such as dependency resolution, data quality checks, and pipeline monitoring.

Example:

A pipeline defining Bronze, Silver, and Gold transformations can be expressed as simple table definitions.

Key Takeaway:

DLT simplifies pipeline development and improves reliability.

Q29. When should teams use Delta Live Tables?

Delta Live Tables is useful when organizations want automated pipeline management with built-in monitoring and data quality features.

It is particularly beneficial for teams that want to reduce operational complexity.

However, for highly customized pipelines, engineers may still prefer traditional Spark jobs.

Key Takeaway:

DLT is best suited for structured data pipelines that benefit from automation and built-in governance.

SECTION 8 — System Design

Q30. Design a production Databricks pipeline for daily sales data.

A production pipeline for sales data typically follows a layered architecture.

First, raw data from transactional systems is ingested into the Bronze layer using batch or streaming ingestion.

Next, transformation pipelines clean and validate the data, producing curated Silver datasets.

Finally, aggregated datasets are created in the Gold layer to support analytics dashboards.

Workflows schedule the pipeline execution and ensure each stage runs in the correct sequence.

Example pipeline flow:

Source system → Bronze raw tables → Silver cleaned tables → Gold analytics tables → BI dashboards.

Key Takeaway:

Well-designed pipelines emphasize reliability, scalability, and data quality.